JavaScript Advanced Challenges - Day 3

Due Date: 2026-03-13 (18:00)

- create the specified file (for example `0_task.js`)
- run with `node <filename>` (Windows/Linux)
- Create a folder named Day3 (mkdir Day3)
- All files should be inside the Day3 folder
- Please note: The Day3 directory should be pushed inside the github repo (js-fundamentals)

Introduction

Welcome to Day 3 of JavaScript Challenges! These 30 challenges build upon Day 2, introducing advanced algorithms, object manipulation, recursion, and complex problem-solving patterns.

How to run a challenge:

```
node challenge_name.js
```

Important Note

⚠ **Continue to solve these challenges yourself.** Building your own solutions is crucial for your learning journey. These challenges are designed to strengthen your problem-solving skills and deepen your understanding of JavaScript fundamentals.

If you get stuck, take these steps instead:

1. Review the Day 1 & Day 2 concepts related to the challenge
2. Break the problem into smaller parts
3. Research JavaScript documentation and tutorials
4. Ask mentors or peers for guidance (not solutions)
5. Debug your code step-by-step

Challenge 0: Deep Clone Object

Objective: Create a deep clone of an object without using JSON methods.

File: `0_deep_clone.js`

Example Inputs & Outputs:

```
Input: {a: 1, b: {c: 2, d: {e: 3}}}
Output: {a: 1, b: {c: 2, d: {e: 3}}}
(with independent memory references)
```

```
Input: {x: 10, y: [1, 2, {z: 3}]}
Output: {x: 10, y: [1, 2, {z: 3}]}
```

Concepts: Objects, Recursion, Reference vs Value

Prototype:

```
/**
 * Creates a deep clone of an object without using JSON methods
 * @param {Object} obj - The object to clone
 * @returns {Object} A deep cloned copy of the object
 */
function deepClone(obj) {
  // Your implementation here
}

// Test cases:
// const original = {a: 1, b: {c: 2, d: {e: 3}}};
// const cloned = deepClone(original);
// cloned.b.d.e = 10;
// console.log(original.b.d.e); // Should still be 3
```

Challenge 1: Longest Substring Without Repeating Characters

Objective: Find the length of the longest substring without repeating characters.

File: `1_longest_substring.js`

Example Inputs & Outputs:

```
Input: "abcabcbb"
Output: 3

Input: "bbbbb"
Output: 1

Input: "pwwkew"
Output: 3
```

Concepts: Strings, Sliding Window, Objects, Loops

Prototype:

```
/**
 * Finds the length of the longest substring without repeating characters
 * @param {string} str - The input string
```

```
 * @returns {number} Length of longest substring without repeating characters
 */
function longestSubstring(str) {
  // Your implementation here
}

// Test cases:
// console.log(longestSubstring("abcabcbb")); // 3
// console.log(longestSubstring("bbbbb")); // 1
// console.log(longestSubstring("pwwkew")); // 3
```

Challenge 2: Group By Property

Objective: Group array of objects by a specific property.

File: `2_group_by_property.js`

Example Inputs & Outputs:

```
Input: [{name: "Alice", age: 25}, {name: "Bob", age: 25}, {name: "Charlie", age:
30}], "age"
Output: {25: [{name: "Alice", age: 25}, {name: "Bob", age: 25}], 30: [{name:
"Charlie", age: 30}]}

Input: [{type: "fruit", name: "apple"}, {type: "vegetable", name: "carrot"}],
"type"
Output: {fruit: [{type: "fruit", name: "apple"}], vegetable: [{type: "vegetable",
name: "carrot"}]}
```

Concepts: Arrays, Objects, Loops, Object properties

Prototype:

```
/**
 * Groups array of objects by a specific property
 * @param {Array<Object>} arr - Array of objects to group
 * @param {string} property - Property name to group by
 * @returns {Object} Object with grouped arrays
 */
function groupByProperty(arr, property) {
  // Your implementation here
}

// Test cases:
// const users = [{name: "Alice", age: 25}, {name: "Bob", age: 25}, {name:
"Charlie", age: 30}];
// console.log(groupByProperty(users, "age"));
```

```
// Output: {25: [{name: "Alice", age: 25}, {name: "Bob", age: 25}], 30: [{name:
"Charlie", age: 30}]}
```

Challenge 3: Prime Number Checker

Objective: Check if a number is prime and find all prime numbers up to n.

File: `3_prime_numbers.js`

Example Inputs & Outputs:

```
Input: 17
Output: true

Input: 20
Output: 20 is not prime. Primes up to 20: [2, 3, 5, 7, 11, 13, 17, 19]

Input: 2
Output: true
```

Concepts: Numbers, Loops, Conditionals, Mathematical logic

Prototype:

```
/**
 * Checks if a number is prime
 * @param {number} n - Number to check
 * @returns {boolean} True if prime, false otherwise
 */
function isPrime(n) {
  // Your implementation here
}

/**
 * Finds all prime numbers up to n
 * @param {number} n - Upper limit
 * @returns {Array<number>} Array of all primes up to n
 */
function getPrimesUpTo(n) {
  // Your implementation here
}

// Test cases:
// console.log(isPrime(17)); // true
// console.log(getPrimesUpTo(20)); // [2, 3, 5, 7, 11, 13, 17, 19]
```

Challenge 4: Word Frequency Counter

Objective: Count the frequency of each word in a text and return sorted results.

File: `4_word_frequency.js`

Example Inputs & Outputs:

```
Input: "hello world hello javascript javascript javascript"
Output: {javascript: 3, hello: 2, world: 1}

Input: "the quick brown fox jumps over the lazy dog"
Output: {the: 2, quick: 1, brown: 1, fox: 1, jumps: 1, over: 1, lazy: 1, dog: 1}
```

Concepts: Strings, Objects, Loops, String methods

Prototype:

```
/**
 * Counts the frequency of each word in a text
 * @param {string} text - The input text
 * @returns {Object} Object with word frequencies
 */
function wordFrequency(text) {
  // Your implementation here
}

// Test cases:
// console.log(wordFrequency("hello world hello javascript javascript
javascript"));
// Output: {javascript: 3, hello: 2, world: 1}
```

Challenge 5: Implement A Debounce Function

Objective: Create a debounce function that delays function execution.

File: `5_debounce.js`

Example Inputs & Outputs:

```
Input: function(x) { console.log(x); }, delay: 1000
Output: Function executes only once after 1 second of calls stopping

function printHello() { console.log("Hello"); }
const debouncedPrint = debounce(printHello, 500);
debouncedPrint(); // Nothing happens
```

```
debouncedPrint(); // Nothing happens
// After 500ms: "Hello" prints once
```

Concepts: Functions, Closures, Timing, Callbacks

Prototype:

```
/**
 * Creates a debounced version of a function
 * @param {Function} func - Function to debounce
 * @param {number} delay - Delay in milliseconds
 * @returns {Function} Debounced function
 */
function debounce(func, delay) {
  // Your implementation here
}

// Test cases:
// function printHello() { console.log("Hello"); }
// const debouncedPrint = debounce(printHello, 500);
// debouncedPrint(); // Call 1
// debouncedPrint(); // Call 2
// debouncedPrint(); // Call 3
// // After 500ms: "Hello" prints once
```

Challenge 6: Transpose Matrix

Objective: Transpose a 2D array (matrix).

File: `6_transpose_matrix.js`

Example Inputs & Outputs:

```
Input: [[1, 2, 3], [4, 5, 6]]
Output: [[1, 4], [2, 5], [3, 6]]

Input: [[1, 2], [3, 4], [5, 6]]
Output: [[1, 3, 5], [2, 4, 6]]
```

Concepts: Arrays, Nested arrays, Loops, 2D data structures

Prototype:

```
/**
 * Transposes a 2D array (matrix)
 * @param {Array<Array<number>>} matrix - 2D array to transpose
```

```
 * @returns {Array<Array<number>>} Transposed matrix
 */
function transposeMatrix(matrix) {
  // Your implementation here
}

// Test cases:
// console.log(transposeMatrix([[1, 2, 3], [4, 5, 6]]));
// Output: [[1, 4], [2, 5], [3, 6]]
```

Challenge 7: Pascal's Triangle

Objective: Generate Pascal's triangle up to n rows.

File: `7_pascals_triangle.js`

Example Inputs & Outputs:

```
Input: 5
Output: [[1], [1, 1], [1, 2, 1], [1, 3, 3, 1], [1, 4, 6, 4, 1]]

Input: 1
Output: [[1]]
```

Concepts: Arrays, Loops, Mathematical patterns, Recursion

Prototype:

```
/**
 * Generates Pascal's triangle up to n rows
 * @param {number} n - Number of rows
 * @returns {Array<Array<number>>} Pascal's triangle
 */
function pascalsTriangle(n) {
  // Your implementation here
}

// Test cases:
// console.log(pascalsTriangle(5));
// Output: [[1], [1, 1], [1, 2, 1], [1, 3, 3, 1], [1, 4, 6, 4, 1]]
```

Challenge 8: Spiral Matrix Traversal

Objective: Traverse a 2D matrix in spiral order (clockwise from outside).

File: `8_spiral_matrix.js`

Example Inputs & Outputs:

```
Input: [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
Output: [1, 2, 3, 6, 9, 8, 7, 4, 5]

Input: [[1, 2], [3, 4]]
Output: [1, 2, 4, 3]
```

Concepts: Arrays, 2D arrays, Loops, Conditionals, Algorithm design

Prototype:

```
/**
 * Traverses a 2D matrix in spiral order
 * @param {Array<Array<number>>} matrix - 2D array to traverse
 * @returns {Array<number>} Elements in spiral order
 */
function spiralTraversal(matrix) {
  // Your implementation here
}

// Test cases:
// console.log(spiralTraversal([[1, 2, 3], [4, 5, 6], [7, 8, 9]]));
// Output: [1, 2, 3, 6, 9, 8, 7, 4, 5]
```

Challenge 9: Build A Simple Calculator (Postfix Notation)

Objective: Evaluate mathematical expressions in postfix notation using a stack concept.

File: `9_postfix_calculator.js`

Example Inputs & Outputs:

```
Input: "3 4 +"
Output: 7

Input: "15 7 1 1 + - / 3 * 2 1 1 + + -"
Output: 5
```

Concepts: Arrays, Loops, Stack concept, Mathematical operations

Prototype:

```
/**
 * Evaluates mathematical expressions in postfix notation
```

```
 * @param {string} expression - Postfix expression with space-separated tokens
 * @returns {number} Result of the expression
 */
function postfixEvaluate(expression) {
  // Your implementation here
}

// Test cases:
// console.log(postfixEvaluate("3 4 +")); // 7
// console.log(postfixEvaluate("15 7 1 1 + - / 3 * 2 1 1 + + -")); // 5
```

Challenge 10: Memoization Function

Objective: Create a function that caches/memoizes results of expensive calculations.

File: `10_memoization.js`

Example Inputs & Outputs:

```
Input: fibonacci(10) with memoization
Output: 55
(Subsequent calls with same input return instantly from cache)

Input: factorial(5)
Output: 120
```

Concepts: Functions, Closures, Objects, Recursion, Performance

Prototype:

```
/**
 * Creates a memoized version of a function
 * @param {Function} func - Function to memoize
 * @returns {Function} Memoized function
 */
function memoize(func) {
  // Your implementation here
}

// Test cases:
// function fibonacci(n) {
//   if (n <= 1) return n;
//   return fibonacci(n - 1) + fibonacci(n - 2);
// }
// const memoFib = memoize(fibonacci);
// console.log(memoFib(10)); // 55 (calculated quickly with memoization)
```

Challenge 11: Intersection of Multiple Arrays

Objective: Find common elements across multiple arrays.

File: `11_multiple_intersection.js`

Example Inputs & Outputs:

```
Input: [[1, 2, 3, 4], [2, 3, 4, 5], [2, 3, 4, 6]]
Output: [2, 3, 4]

Input: [[1, 2], [2, 3], [3, 4]]
Output: []
```

Concepts: Arrays, Loops, Nested loops, Sets or Objects

Prototype:

```javascript
/**
 * Finds common elements across multiple arrays
 * @param {Array<Array<number>>} arrays - Array of arrays
 * @returns {Array<number>} Common elements
 */
function findIntersection(arrays) {
  // Your implementation here
}

// Test cases:
// console.log(findIntersection([[1, 2, 3, 4], [2, 3, 4, 5], [2, 3, 4, 6]]));
// Output: [2, 3, 4]
```

Challenge 12: Convert Decimal To Base N

Objective: Convert a decimal number to any base (2 to 36).

File: `12_decimal_to_base.js`

Example Inputs & Outputs:

```
Input: 255, 2
Output: "11111111"

Input: 255, 16
Output: "FF"

Input: 100, 8
Output: "144"
```

Concepts: Numbers, String building, Mathematical operations, Loops

Prototype:

```
/**
 * Converts a decimal number to any base (2 to 36)
 * @param {number} decimal - Decimal number to convert
 * @param {number} base - Target base (2-36)
 * @returns {string} Number in target base
 */
function decimalToBase(decimal, base) {
  // Your implementation here
}

// Test cases:
// console.log(decimalToBase(255, 2)); // "11111111"
// console.log(decimalToBase(255, 16)); // "FF"
// console.log(decimalToBase(100, 8)); // "144"
```

Challenge 13: Permutations of A String

Objective: Generate all permutations of a string.

File: `13_string_permutations.js`

Example Inputs & Outputs:

```
Input: "ABC"
Output: ["ABC", "ACB", "BAC", "BCA", "CAB", "CBA"]

Input: "AB"
Output: ["AB", "BA"]
```

Concepts: Strings, Recursion, Arrays, Algorithm design

Prototype:

```
/**
 * Generates all permutations of a string
 * @param {string} str - Input string
 * @returns {Array<string>} All permutations
 */
function getPermutations(str) {
  // Your implementation here
}
```

```
// Test cases:
// console.log(getPermutations("ABC"));
// Output: ["ABC", "ACB", "BAC", "BCA", "CAB", "CBA"]
```

Challenge 14: Longest Increasing Subsequence

Objective: Find the length of the longest increasing subsequence in an array.

File: `14_longest_increasing.js`

Example Inputs & Outputs:

```
Input: [10, 9, 2, 5, 3, 7, 101, 18]
Output: 4
(subsequence: [2, 3, 7, 101] or [2, 3, 7, 18])

Input: [0, 1, 0, 4, 4, 4, 3, 2, 1]
Output: 3
```

Concepts: Arrays, Loops, Dynamic Programming concepts, Conditionals

Prototype:

```
/**
 * Finds the length of the longest increasing subsequence
 * @param {Array<number>} arr - Array of numbers
 * @returns {number} Length of LIS
 */
function longestIncreasingSubsequence(arr) {
  // Your implementation here
}

// Test cases:
// console.log(longestIncreasingSubsequence([10, 9, 2, 5, 3, 7, 101, 18])); // 4
// console.log(longestIncreasingSubsequence([0, 1, 0, 4, 4, 4, 3, 2, 1])); // 3
```

Challenge 15: Object Deep Freeze

Objective: Recursively freeze an object to prevent any modifications.

File: `15_deep_freeze.js`

Example Inputs & Outputs:

```
Input: {a: 1, b: {c: 2}}
Output: {a: 1, b: {c: 2}}
(Any attempt to modify raises an error in strict mode)

obj.a = 5; // Error: Cannot assign to read-only property 'a'
```

Concepts: Objects, Recursion, Object methods, Immutability

Prototype:

```
/**
 * Recursively freezes an object to prevent modifications
 * @param {Object} obj - Object to freeze
 * @returns {Object} Frozen object
 */
function deepFreeze(obj) {
  // Your implementation here
}

// Test cases:
// const frozen = deepFreeze({a: 1, b: {c: 2}});
// frozen.a = 5; // Error in strict mode
// frozen.b.c = 10; // Error in strict mode
```

Challenge 16: Replace Nested Property In Object

Objective: Replace a property value in a nested object without mutating the original.

File: `16_replace_nested.js`

Example Inputs & Outputs:

```
Input: {a: 1, b: {c: {d: 2}}}, ["b", "c", "d"], 10
Output: {a: 1, b: {c: {d: 10}}}
(Original object unchanged)

Input: {name: "John", address: {city: "NYC"}}, ["address", "city"], "LA"
Output: {name: "John", address: {city: "LA"}}
```

Concepts: Objects, Recursion, Immutability, Array methods, Spread operator

Prototype:

```
/**
 * Replaces a property value in a nested object without mutating original
 * @param {Object} obj - Original object
```

```
 * @param {Array<string>} path - Path to the property (array of keys)
 * @param {*} newValue - New value to set
 * @returns {Object} New object with updated value
 */
function replaceNestedProperty(obj, path, newValue) {
  // Your implementation here
}

// Test cases:
// const original = {a: 1, b: {c: {d: 2}}};
// const updated = replaceNestedProperty(original, ["b", "c", "d"], 10);
// console.log(updated); // {a: 1, b: {c: {d: 10}}}
// console.log(original); // Unchanged: {a: 1, b: {c: {d: 2}}}
```

Challenge 17: Regex Pattern Matcher

Objective: Create a simple function to validate email and phone number patterns.

File: `17_regex_patterns.js`

Example Inputs & Outputs:

```
Input: "test@example.com"
Output: {email: true, phone: false}

Input: "555-123-4567"
Output: {email: false, phone: true}

Input: "john.doe@domain.co.uk"
Output: {email: true, phone: false}
```

Concepts: Regular Expressions, Strings, Pattern matching, Conditionals

Prototype:

```
/**
 * Validates email and phone number patterns
 * @param {string} input - String to validate
 * @returns {Object} Object with email and phone validation results
 */
function validatePatterns(input) {
  // Your implementation here
  // Returns: {email: boolean, phone: boolean}
}

// Test cases:
// console.log(validatePatterns("test@example.com")); // {email: true, phone:
```

```
false}
// console.log(validatePatterns("555-123-4567")); // {email: false, phone: true}
```

Challenge 18: Implement Promise.All

Objective: Create a function that mimics Promise.all() functionality.

File: `18_promise_all.js`

Example Inputs & Outputs:

```
Input: [Promise.resolve(1), Promise.resolve(2), Promise.resolve(3)]
Output: Promise resolves with [1, 2, 3]

Input: [Promise.resolve(10), Promise.reject("Error")]
Output: Promise rejects with "Error"
```

Concepts: Promises, Asynchronous programming, Arrays, Error handling

Prototype:

```
/**
 * Mimics Promise.all() functionality
 * @param {Array<Promise>} promises - Array of promises
 * @returns {Promise} Promise that resolves with array of results or rejects with
first error
 */
function promiseAll(promises) {
  // Your implementation here
}

// Test cases:
// promiseAll([Promise.resolve(1), Promise.resolve(2), Promise.resolve(3)])
//   .then(result => console.log(result)); // [1, 2, 3]
// promiseAll([Promise.resolve(10), Promise.reject("Error")])
//   .catch(error => console.log(error)); // "Error"
```

Challenge 19: Currying Function

Objective: Convert a regular function into a curried function.

File: `19_currying.js`

Example Inputs & Outputs:

```
Input: function add(a, b, c) { return a + b + c; }
Output: curry(add)(1)(2)(3) = 6
         curry(add)(1, 2)(3) = 6

Input: function multiply(x, y) { return x * y; }
Output: curry(multiply)(5)(4) = 20
```

Concepts: Functions, Closures, Higher-order functions, Function composition

Prototype:

```
/**
 * Converts a regular function into a curried function
 * @param {Function} func - Function to curry
 * @returns {Function} Curried version of the function
 */
function curry(func) {
  // Your implementation here
}

// Test cases:
// function add(a, b, c) { return a + b + c; }
// const curriedAdd = curry(add);
// console.log(curriedAdd(1)(2)(3)); // 6
// console.log(curriedAdd(1, 2)(3)); // 6
```

Challenge 20: Dynamic Programming - Staircase

Objective: Find the number of ways to climb a staircase with 1 or 2 steps at a time.

File: `20_staircase_ways.js`

Example Inputs & Outputs:

```
Input: 4
Output: 5
(1+1+1+1, 1+1+2, 1+2+1, 2+1+1, 2+2)

Input: 3
Output: 3
(1+1+1, 1+2, 2+1)
```

Concepts: Recursion, Dynamic Programming, Loops, Conditionals

Prototype:

```
/**
 * Finds the number of ways to climb a staircase with 1 or 2 steps at a time
 * @param {number} n - Number of stairs
 * @returns {number} Number of ways to climb
 */
function countWaysToclimbStairs(n) {
  // Your implementation here
}

// Test cases:
// console.log(countWaysToclimbStairs(4)); // 5 (1+1+1+1, 1+1+2, 1+2+1, 2+1+1,
2+2)
// console.log(countWaysToclimbStairs(3)); // 3 (1+1+1, 1+2, 2+1)
```

Challenge 21: Implement A Simple Event Emitter

Objective: Create an event emitter class with on, emit, and off methods.

File: `21_event_emitter.js`

Example Inputs & Outputs:

```
const emitter = new EventEmitter();
emitter.on('greet', (name) => console.log(`Hello ${name}`));
emitter.emit('greet', 'Alice'); // Output: Hello Alice

emitter.on('data', (value) => console.log(value));
emitter.emit('data', 42); // Output: 42
emitter.off('data');
emitter.emit('data', 100); // Nothing happens
```

Concepts: Classes, Objects, Callbacks, Array methods, Higher-order functions

Prototype:

```
/**
 * Event emitter class for publishing and subscribing to events
 */
class EventEmitter {
  /**
   * Constructor
   */
  constructor() {
    // Your implementation here
  }

  /**
   * Subscribe to an event
```

```
     * @param {string} event - Event name
     * @param {Function} callback - Callback function
     */
    on(event, callback) {
      // Your implementation here
    }

    /**
     * Emit an event with data
     * @param {string} event - Event name
     * @param {...*} args - Arguments to pass to callbacks
     */
    emit(event, ...args) {
      // Your implementation here
    }

    /**
     * Unsubscribe from an event
     * @param {string} event - Event name
     * @param {Function} callback - Callback to remove
     */
    off(event, callback) {
      // Your implementation here
    }
  }

  // Test cases:
  // const emitter = new EventEmitter();
  // emitter.on('greet', (name) => console.log(`Hello ${name}`));
  // emitter.emit('greet', 'Alice'); // Output: Hello Alice
```

Challenge 22: Find All Unique Paths In Matrix

Objective: Count the number of unique paths from top-left to bottom-right in a matrix.

File: `22_unique_paths.js`

Example Inputs & Outputs:

```
Input: 3x3 grid
Output: 6

Input: 2x2 grid
Output: 2

Input: 1x1 grid
Output: 1
```

Concepts: Dynamic Programming, Recursion, 2D arrays, Loops, Mathematical combinations

Prototype:

```
/**
 * Counts the number of unique paths from top-left to bottom-right in a matrix
 * @param {number} rows - Number of rows
 * @param {number} cols - Number of columns
 * @returns {number} Number of unique paths
 */
function uniquePaths(rows, cols) {
  // Your implementation here
}

// Test cases:
// console.log(uniquePaths(3, 3)); // 6
// console.log(uniquePaths(2, 2)); // 2
// console.log(uniquePaths(1, 1)); // 1
```

Challenge 23: Map and Filter Implementation

Objective: Implement custom map and filter functions without using built-in methods.

File: `23_map_filter.js`

Example Inputs & Outputs:

```
myMap([1, 2, 3], x => x * 2) = [2, 4, 6]
myFilter([1, 2, 3, 4], x => x % 2 === 0) = [2, 4]

myMap(["a", "b"], x => x.toUpperCase()) = ["A", "B"]
myFilter(["apple", "banana", "cherry"], x => x.length > 5) = ["banana", "cherry"]
```

Concepts: Array methods, Functions, Callbacks, Higher-order functions, Loops

Prototype:

```
/**
 * Implements custom map function
 * @param {Array} arr - Array to map
 * @param {Function} callback - Transform function
 * @returns {Array} Mapped array
 */
function myMap(arr, callback) {
  // Your implementation here
}

/**
 * Implements custom filter function
```

```
 * @param {Array} arr - Array to filter
 * @param {Function} predicate - Test function
 * @returns {Array} Filtered array
 */
function myFilter(arr, predicate) {
  // Your implementation here
}

// Test cases:
// console.log(myMap([1, 2, 3], x => x * 2)); // [2, 4, 6]
// console.log(myFilter([1, 2, 3, 4], x => x % 2 === 0)); // [2, 4]
```

Challenge 24: Binary Search Implementation

Objective: Implement binary search to find a target value in a sorted array.

File: `24_binary_search.js`

Example Inputs & Outputs:

```
Input: [1, 3, 5, 7, 9, 11], 7
Output: 3 (index)

Input: [2, 4, 6, 8, 10], 5
Output: -1 (not found)

Input: [10, 20, 30, 40], 10
Output: 0 (index)
```

Concepts: Algorithms, Binary search, Loops, Conditionals, Array indexing, Recursion

Prototype:

```
/**
 * Implements binary search to find target in sorted array
 * @param {Array<number>} arr - Sorted array
 * @param {number} target - Value to search for
 * @returns {number} Index of target or -1 if not found
 */
function binarySearch(arr, target) {
  // Your implementation here
}

// Test cases:
// console.log(binarySearch([1, 3, 5, 7, 9, 11], 7)); // 3
// console.log(binarySearch([2, 4, 6, 8, 10], 5)); // -1
// console.log(binarySearch([10, 20, 30, 40], 10)); // 0
```

Challenge 25: Merge K Sorted Arrays

Objective: Merge multiple sorted arrays into a single sorted array.

File: `25_merge_k_arrays.js`

Example Inputs & Outputs:

```
Input: [[1, 4, 5], [1, 3, 4], [2, 6]]
Output: [1, 1, 2, 3, 4, 4, 5, 6]

Input: [[1, 2], [3, 4], [5, 6]]
Output: [1, 2, 3, 4, 5, 6]
```

Concepts: Arrays, Nested arrays, Loops, Sorting algorithms, Pointer technique

Prototype:

```
/**
 * Merges multiple sorted arrays into a single sorted array
 * @param {Array<Array<number>>} arrays - Array of sorted arrays
 * @returns {Array<number>} Merged and sorted array
 */
function mergeKSortedArrays(arrays) {
  // Your implementation here
}

// Test cases:
// console.log(mergeKSortedArrays([[1, 4, 5], [1, 3, 4], [2, 6]]));
// Output: [1, 1, 2, 3, 4, 4, 5, 6]
```

Challenge 26: Object to Array and Array to Object Conversion

Objective: Convert between object and array formats with different structures.

File: `26_conversion.js`

Example Inputs & Outputs:

```
Input: {a: 1, b: 2, c: 3}
Output: [["a", 1], ["b", 2], ["c", 3]]

Input: [[1, "apple"], [2, "banana"]]
Output: {1: "apple", 2: "banana"}
```

```
Input: {name: "John", age: 30}
Output: [{key: "name", value: "John"}, {key: "age", value: 30}]
```

Concepts: Objects, Arrays, Loops, Object methods, Transformation

Prototype:

```
/**
 * Converts object to array of [key, value] pairs
 * @param {Object} obj - Object to convert
 * @returns {Array<Array>} Array of [key, value] pairs
 */
function objectToArray(obj) {
  // Your implementation here
}

/**
 * Converts array of [key, value] pairs to object
 * @param {Array<Array>} arr - Array of [key, value] pairs
 * @returns {Object} Resulting object
 */
function arrayToObject(arr) {
  // Your implementation here
}

/**
 * Converts object to array of {key, value} objects
 * @param {Object} obj - Object to convert
 * @returns {Array<Object>} Array of {key, value} objects
 */
function objectToKeyValueArray(obj) {
  // Your implementation here
}

// Test cases:
// console.log(objectToArray({a: 1, b: 2})); // [["a", 1], ["b", 2]]
// console.log(arrayToObject([[1, "apple"], [2, "banana"]])); // {1: "apple", 2:
"banana"}
```

Challenge 27: Find Consecutive Numbers In Array

Objective: Find the length of the longest sequence of consecutive numbers in an unsorted array.

File: `27_consecutive.js`

Example Inputs & Outputs:

```
Input: [100, 4, 200, 1, 3, 2]
Output: 4 (sequence: 1, 2, 3, 4)

Input: [9, 1,4,7, 3,2,8,5, 6]
Output: 9 (sequence: 1-9)

Input: [0, 3, 7, 2, 5, 8, 4, 6, 1]
Output: 9
```

Concepts: Arrays, Objects/Sets, Loops, Sorting, Algorithm optimization

Prototype:

```
/**
 * Finds the length of longest sequence of consecutive numbers
 * @param {Array<number>} arr - Unsorted array of numbers
 * @returns {number} Length of longest consecutive sequence
 */
function longestConsecutiveSequence(arr) {
  // Your implementation here
}

// Test cases:
// console.log(longestConsecutiveSequence([100, 4, 200, 1, 3, 2])); // 4 (1, 2, 3,
4)
// console.log(longestConsecutiveSequence([9, 1,4,7, 3,2,8,5,6])); // 9
```

Challenge 28: Implement Throttle Function

Objective: Create a throttle function that limits execution frequency.

File: `28_throttle.js`

Example Inputs & Outputs:

```
const throttledFunc = throttle(() => console.log("fired"), 1000);
throttledFunc(); // Executes immediately
throttledFunc(); // Ignored
throttledFunc(); // Ignored
// After 1 second: throttledFunc() can execute again

window.addEventListener('scroll', throttle(() => {
  console.log('Scroll event');
}, 1000)); // Fires at most once every 1000ms
```

Concepts: Functions, Closures, Timing, Performance optimization, Callbacks

Prototype:

```
/**
 * Creates a throttled version of a function
 * @param {Function} func - Function to throttle
 * @param {number} limit - Time limit in milliseconds
 * @returns {Function} Throttled function
 */
function throttle(func, limit) {
  // Your implementation here
}

// Test cases:
// const throttledFunc = throttle(() => console.log("fired"), 1000);
// throttledFunc(); // Executes immediately
// throttledFunc(); // Ignored
// throttledFunc(); // Ignored
// // After 1 second: throttledFunc() can execute again
```

Challenge 29: Implement Reduce Function

Objective: Implement a custom reduce function and solve problems using it.

File: `29_reduce.js`

Example Inputs & Outputs:

```
myReduce([1, 2, 3, 4], (sum, val) => sum + val, 0) = 10

myReduce(["a", "b", "c"], (str, val) => str + val, "") = "abc"

myReduce([1, 2, 3], (sum, val) => sum + val) = 6

myReduce([{a: 1}, {b: 2}], (obj, val) => ({...obj, ...val}), {}) = {a: 1, b: 2}
```

Concepts: Array methods, Higher-order functions, Callbacks, Accumulation, Spread operator

Prototype:

```
/**
 * Implements custom reduce function
 * @param {Array} arr - Array to reduce
 * @param {Function} callback - Accumulator function
 * @param {*} initialValue - Initial accumulator value
 * @returns {*} Final accumulated value
 */
function myReduce(arr, callback, initialValue) {
```

```
  // Your implementation here
}

// Test cases:
// console.log(myReduce([1, 2, 3, 4], (sum, val) => sum + val, 0)); // 10
// console.log(myReduce(["a", "b", "c"], (str, val) => str + val, "")); // "abc"
// console.log(myReduce([1, 2, 3], (sum, val) => sum + val)); // 6 (no initial
value)
```

Learning Progression

1. **Advanced Object/Data Structure Operations (0, 2, 6, 11, 15, 16, 26):** Complex object and array manipulation
2. **Algorithm Design & Optimization (1, 5, 10, 14, 22, 24, 25, 27, 28):** Performance and algorithm patterns
3. **Mathematical & Logical Challenges (3, 7, 8, 9, 12, 13, 20, 23):** Complex problem-solving
4. **Advanced String & Pattern Matching (4, 13, 17):** Text processing at higher levels
5. **Functional Programming & Async (18, 19, 21, 29):** Higher-order functions, promises, and event handling

Tips for Success

- Build upon the Day 2 concepts and patterns
- Use helper functions to break down complex problems
- Practice recursion when appropriate
- Test edge cases (empty inputs, single elements, etc.)
- Comment your code to explain your logic and approach
- Try different approaches and compare performance
- Consider using modern JavaScript features (spread operator, destructuring, etc.)
- Think about time and space complexity

Keep Pushing! 🚀